

Paradigmi di Programmazione - A.A. 2022-23

Esame Scritto del 15/06/2023

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione applicando la strategia **call-by-value** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita. Ripetere l'esercizio applicando la strategia **call-by-name**.

$$(\lambda f. \lambda x. x)((\lambda f. f f)(\lambda x. x x))$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Call-by-value:

$$\begin{aligned} & (\lambda f. \lambda x. x)((\lambda f. f f)(\lambda x. x x)) \\ \rightarrow & \quad \underline{(\lambda f. \lambda x. x)((\lambda x. x x)(\lambda x. x x))} \\ \rightarrow & \quad \underline{(\lambda f. \lambda x. x)((\lambda x. x x)(\lambda x. x x))} \\ \rightarrow & \quad \dots \text{derivazione infinita} \end{aligned}$$

Call-by-name:

$$\begin{aligned} & \underline{(\lambda f. \lambda x. x)((\lambda f. f f)(\lambda x. x x))} \\ \rightarrow & \quad \lambda x. x \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
fun x -> fun y -> match x with
| [] -> []
| z::[] -> z::z::[]
| (z1,z2)::x' -> (z1+y,z2)::x';;
```

SOLUZIONE:

Struttura del tipo:

`X -> Y --> RIS`

Uso per convenzione `X,Y` come variabili di tipo per i parametri `x,y`, `RIS` come variabile di tipo del risultato, e `A,B,C,...` come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
X = A*B list      (da pattern matching)
A = int           (da terzo caso del pattern matching)
Y = int           (da terzo caso del pattern matching)
RIS = int*B list  (da terzo caso del pattern matching)
```

Ne consegue:

```
X = int*B list
Y = int
RIS = int*B list
```

Tipo inferito:

```
(int * 'a) list -> int -> (int * 'a) list
```

Esercizio 3 [Punti 7]

Definire in OCaml le funzioni `stessalunghezza` e `separa` con i seguenti tipi

```
stessalunghezza : 'a list list -> bool
separa : ('a * 'b) list list -> 'a list list * 'b list list
```

tali che:

- `stessalunghezza` prende una lista di liste `lis` e verifica se tutte le liste contenute in `lis` hanno la stessa lunghezza. Ad esempio:

```
stessalunghezza [[1;2;3];[4;5;6];[7;8;9]] (* restituisce true *)
stessalunghezza [['a';'b'];['c']]          (* restituisce false *)
```

- `separa` prende una lista di liste di coppie `lis` e restituisce una coppia di liste di liste (`lis1,lis2`) tale che ogni lista di coppie $[(a_1,b_1); \dots; (a_N,b_N)]$ in `lis` viene trasformata in due liste, una con i primi elementi $[a_1; \dots; a_N]$ e una con i secondi elementi $[b_1; \dots; b_N]$, e le due liste vengono inserite rispettivamente in `lis1` e `lis2`. Ad esempio:

```
separa [[(1,'a');(2,'b');(3,'c')];[(4,'d');(5,'e')]]

(* restituisce ([ [1;2;3];[4;5] ], [ ['a','b','c'], ['d','e'] ] *)
```

SOLUZIONE:

Una possibile soluzione:

```
(* fst e snd in realtà sono già definite in OCaml *)
let fst (x,y) = x;;
let snd (x,y) = y;;

let separa lis =
List.map (fun l -> List.map (fst) l) lis , List.map (fun l -> List.map (snd) l) lis;;
```

Altra soluzione:

```
let rec separa lis =
  let rec aux l =
    match l with
    | [] -> ([],[])
    | (x,y)::lis' -> let (l1,l2) = aux lis' in (x::l1,y::l2)
  in
  match lis with
  | [] -> ([],[])
  | l::lis' -> let (lis1,lis2) = separate2 lis' in
    let (l1,l2) = aux l in
    (l1::lis1,l2::lis2);;
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con il costrutto **NaryFun** per la definizione di *funzioni anonime n-arie* (non Curryed), ossia che prevedono un numero arbitrario di parametri formali. Queste nuove funzioni saranno applicate tramite un nuovo costrutto di applicazione che, data una lista di parametri attuali con una lunghezza pari al numero di parametri formali, lega ogni parametro formale al corrispondente parametro attuale per creare l'ambiente in cui viene eseguito il corpo della funzione. Ad esempio (usando una sintassi simile a OCaml):

```
nary_fun [x,y] -> x+y      (* definisce una funzione che prende x e y, e ne fa la somma *)

(nary_fun [x,y] -> x+y) [3,2]  (* applica la funzione di somma a 3 e 2 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.

SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | NAryFun of ide list * exp
  | NAryApply of exp * exp list

type evT = ...
  | NAryClosure of ide list * exp * evT env

let rec eval e s = match e with
  ...
  | NAryFun (arg_list, ebody) -> NAryClosure (arg_list,ebody,s)
  | NAryApply(eF,elist) ->
    let fclosure = eval eF s in
    (match fclosure with
      | NAryClosure (arg_list,body,fDecEnv) ->
        let val_list = List.map (fun e -> eval e s) elist in
        let rec bindall al vl env =
          (match al,vl with
            | [],[] -> env
            | x::al',v::vl' -> bindall al' vl' (bind env x v)
            | _ -> failwith "numero argomenti errato"
          )
        in
        eval body (bindall arg_list val_list fDecEnv)
      | _ -> failwith "Type error"
    )
  )
```